

progetto asd

PIETRO FUBINI

May 2026

1 Come gestire i dati

ogni nodo(formato dal nome nodo e la funzione di hashing universale) ha la lista di adiacenza lasciata con hashing universale

la lista di adiacenze è formata da coppie, con il vicino e il peso

i nodi sono a loro volta lasciati in una tabella per cercarli bene

1.1 funzioni utili

1.1.1 albero puntato

input: nodo partenza e arrivo, cap sulla lunghezza

processo: prende un nodo e scorre tutta la sua lista di adiacenza, aggiungendo tutti i nodi con meno della distanza max, ogni nodo aggiunto è inserito nella terna:

- nome nodo
- nodo babbo
- insieme lasciato di nodi già percorsi

continua questo processo se trova il nodo di arrivo si ferma, se non lo trova usa cancellanodo in un albero dal nodo iniziale di questo ramo

output: un albero con tutti i percorsi banalmente ogni ramo fogliato è un percorso diverso

1.1.2 cancellanodo in un albero

input: nodo

procedimento: passa la funzione a tutti i suoi figli e poi quando torna a lui elimina il nodo

output: albero senza il nodo e tutti i suoi figli

1.1.3 lista adiacenza

prende in input un nodo e ritorna un vettore di coppie con la sua lista di adiacenza con i pesi

1.1.4 aggiungi nodo

aggiunge il nodo, randomizzando una hash universale, prende in input un int(i vari nodi sono degli int) e per l'albero dereferenziato

1.1.5 aggiungi arco

input: nodo 1 e nodo 2

processo: va in nodo uno e cerca nella lista di adiacenza con la funzione di hasching il nodo 2 se non ce lo aggiunge con il peso 1 altrimenti aumenta di 1 il peso fa lo al contrario,

in teoria è $O(1)$

output: è aggiunto in entrambe delle liste di adiacenza in più uno su quell'arco

2 da .all-paths a struttura di grafo

prende il testo linea linea e sputa il grafo. ogni volta prende 2 nomi e aggiunge o somma uno alla difficoltà del ramo
deve ritornare il percorso più lungo

3 funzioni utili

3.1 conta foglie

input: nodo

processo: ricorsivo:

- passo base: il nodo non ha figli ritorna 1;
- passo induttivo il nodo ha figli, ritorna il risultato dei figli

output: numero di foglie

4 minimo percorso tra 2

greedy, ricorsivo.

prende il nodo e (controlla se ce l'ultimo nodo) lascia i suoi vicini con la funzione che lascia in base alla lunghezza, e poi lascia universalmente nella cardinalità dei vicini, e poi fa la ricorsione(sul primo della lista relativa al percorso più leggero), passandosi ogni volta che fa il numero più alto preso.

i casi doppi: quando prendi una lista di adiacenza guarda il peso del suo arco

e controlla nei pesi prima se ce, se ce non lo aggiungere altrimenti aggiungilo e togliilo dalla lista prima (se ha senso)

il passaggio ricorsivo è ricerca che ci mette $O(1)$ aggiunta